

2

Modelos de processo

Conceitos-chave

modelo de processo evolucionário

modelo de processo genérico

avaliação de processos

fluxo de processo

aperfeiçoamento de processos

padrões de processo

prototipação

modelo espiral

conjunto de tarefas

Processo Unificado

modelo cascata

Construir *software* é um processo de aprendizado social iterativo, e o resultado, algo que Baetjer [Bae98] denominaria “capital de *software*”, é a incorporação do conhecimento coletado, filtrado e organizado à medida que se desenvolve o processo.

Mas, do ponto de vista técnico, o que é exatamente um processo de *software*? No contexto deste livro, definimos *processo de software* como uma metodologia para as atividades, ações e tarefas necessárias para desenvolver um *software* de alta qualidade. “Processo” é sinônimo de “engenharia de *software*”? A resposta é “sim e não”. Um processo de *software* define a abordagem adotada conforme um *software* é elaborado pela engenharia. Entretanto, a engenharia de *software* também engloba tecnologias que fazem parte do processo – métodos técnicos e ferramentas automatizadas.

Mais importante, a engenharia de *software* é realizada por pessoas criativas e com amplo conhecimento, e que devem adaptar um processo de *software* maduro de modo que fique adequado aos produtos desenvolvidos e às demandas de seu mercado.



Panorama

O que é? Quando se elabora um produto ou sistema, é importante seguir uma série de passos previsíveis (um roteiro) que ajude a criar um produto de alta qualidade e dentro do prazo. O roteiro é denominado “processo de software”.

Quem realiza? Engenheiros de *software* adaptam o processo às suas necessidades e então o seguem. Os solicitantes do *software* também têm um papel a desempenhar no processo de definição, construção e teste do *software*.

Por que é importante? O processo propicia estabilidade, controle e organização para uma atividade que pode tornar-se bastante caótica. Entretanto, um processo de engenharia de *software* moderno deve ser “ágil”. Ele deve demandar apenas atividades, controles e produtos de trabalho que sejam apropriados para a equipe do projeto e para o produto a ser gerado.

Quais são as etapas envolvidas? O processo adotado depende do *software* a ser desenvolvido. Um processo pode ser adequado para um *software* de uma aeronave, mas pode não funcionar bem para a criação de um aplicativo móvel ou de um videogame.

Qual é o artefato? Os artefatos são os programas, os documentos e os dados produzidos pelas atividades e tarefas incluídas no processo.

Como garantir que o trabalho foi realizado corretamente? A qualidade, o cumprimento de prazos e a viabilidade em longo prazo do produto que se desenvolve são os melhores indicadores do sucesso do processo utilizado.

2.1 Um modelo de processo genérico

No Capítulo 1, *processo* foi definido como um conjunto de atividades de trabalho, ações e tarefas realizadas quando algum artefato de *software* deve ser criado. Cada uma dessas atividades, ações e tarefas se aloca dentro de uma metodologia ou modelo que determina sua relação com o processo e uma com a outra.

O processo de *software* está representado esquematicamente na Figura 2.1. De acordo com a figura, cada atividade metodológica é composta por um conjunto de ações de engenharia de *software*. Cada ação é definida por um *conjunto de tarefas*, o qual identifica as tarefas de trabalho que devem ser completadas, os artefatos de *software* que serão produzidos, os fatores de garantia da qualidade que serão exigidos e os marcos utilizados para indicar progresso.

Processo de *software*

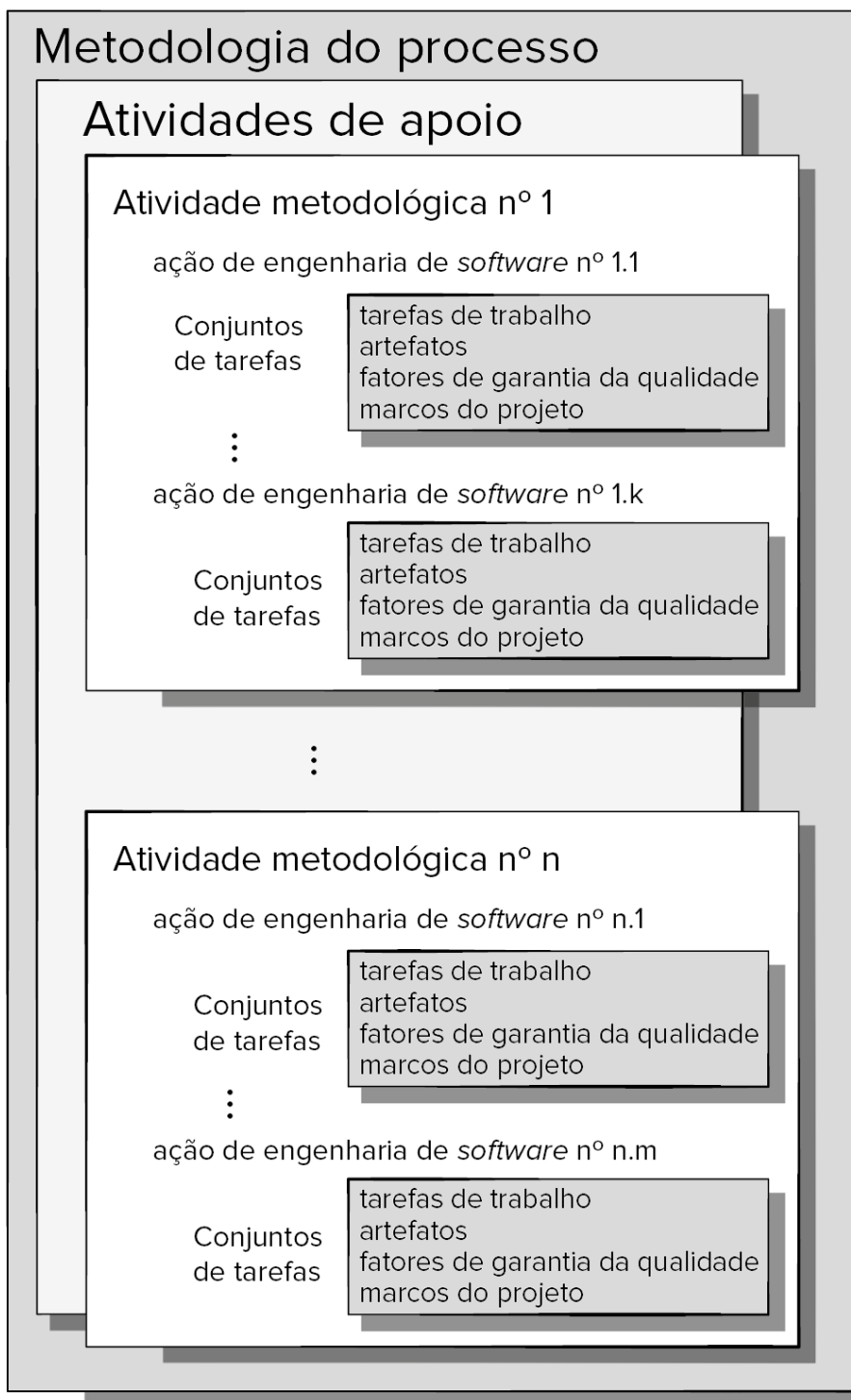


Figura 2.1

Uma metodologia do processo de *software*.

Como discutido no Capítulo 1, uma metodologia de processo genérica para engenharia de *software* estabelece cinco atividades metodológicas: **comunicação, planejamento, modelagem, construção e entrega**. Além disso, um conjunto de atividades de apoio é aplicado ao longo do processo, como o acompanhamento e o controle do projeto, a administração de riscos, a garantia da qualidade, o gerenciamento das configurações, as revisões técnicas, entre outras.

Um aspecto importante do processo de *software* ainda não foi discutido. Este aspecto – chamado de *fluxo de processo* – descreve como são organizadas as atividades metodológicas, bem como as ações e tarefas que ocorrem dentro de cada atividade em relação à sequência e ao tempo, como ilustrado na Figura 2.2.

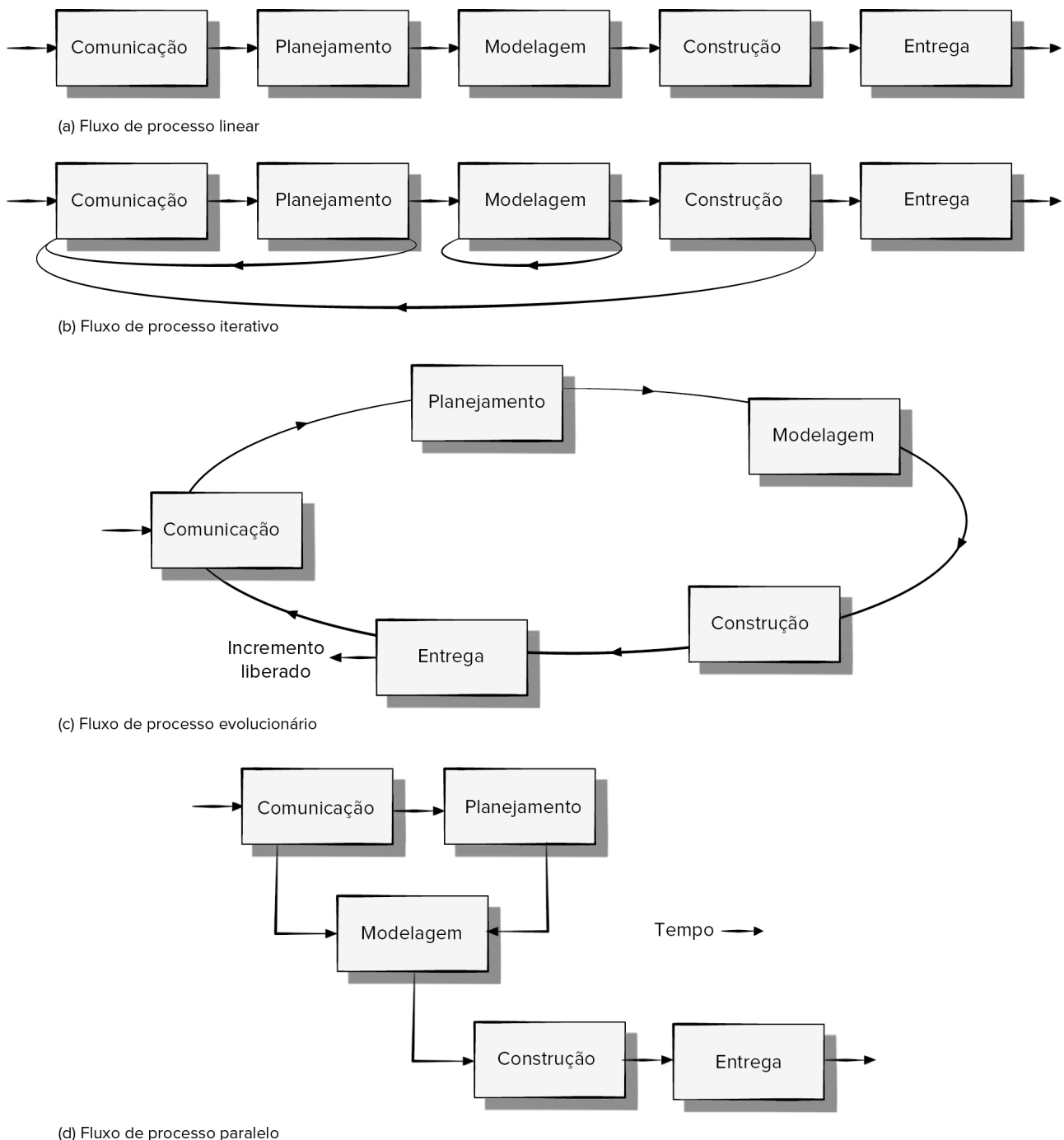


Figura 2.2
Fluxo de processo.

Um *fluxo de processo linear* executa cada uma das cinco atividades metodológicas em sequência, começando com a comunicação e culminando com a entrega (Figura 2.2a). Um *fluxo de processo iterativo* repete uma ou mais das atividades antes de prosseguir para a seguinte (Figura 2.2b). Um *fluxo de processo evolucionário* executa as atividades de forma “circular”. Cada volta pelas cinco atividades conduz a uma versão mais completa do *software* (Figura 2.2c). Um *fluxo de processo paralelo* (Figura 2.2d) executa uma ou mais atividades em paralelo com outras (p. ex., a modelagem para um aspecto do *software* poderia ser executada em paralelo com a construção de outro aspecto do *software*).

2.2 Definição de uma atividade metodológica

Embora cinco atividades metodológicas tenham sido descritas e tenha-se fornecido uma definição básica de cada uma delas no Capítulo 1, uma equipe de *software* precisa de muito mais informações antes de poder executar qualquer uma das atividades como parte do processo de *software*. Assim, enfrenta-se uma questão-chave: *Quais ações são apropriadas para uma atividade metodológica, uma vez fornecidos a natureza do problema a ser solucionado, as características das pessoas que farão o trabalho e os envolvidos no projeto?*

Para um pequeno projeto de *software* solicitado por uma única pessoa (em um local distante) com requisitos simples e objetivos, a atividade de **comunicação** pode se resumir a pouco mais de um

telefonema ou *e-mail* para o envolvido. Portanto, a única ação necessária é uma *conversa telefônica*, e as tarefas de trabalho (o *conjunto de tarefas*) que essa ação envolve são:

5. Contatar o envolvido via telefone.
8. Discutir os requisitos e gerar anotações.
7. Organizar as anotações em uma breve relação de requisitos, por escrito.
0. Enviar um *e-mail* para o envolvido para revisão e aprovação.

Se o projeto fosse consideravelmente mais complexo, com muitos envolvidos, cada qual com um conjunto de requisitos diferentes (por vezes conflitantes), a atividade de comunicação poderia ter seis ações distintas: *concepção*, *levantamento*, *elaboração*, *negociação*, *especificação* e *validação*. Cada uma dessas ações de engenharia de *software* conteria muitas tarefas de trabalho e uma série de diferentes artefatos.

2.3 Identificação de um conjunto de tarefas

Voltando novamente à Figura 2.1, cada ação de engenharia de *software* (p. ex., *levantamento*, uma ação associada à atividade de **comunicação**) pode ser representada por vários e diferentes *conjuntos de tarefas* – constituídos por uma gama de tarefas de trabalho de engenharia de *software*, artefatos relacionados, fatores de garantia da qualidade e marcos do projeto. Projetos diferentes exigem conjuntos de tarefas diferentes. Deve-se escolher um conjunto de tarefas mais adequado às necessidades do projeto e às características da equipe. Isso significa que uma ação de engenharia de *software* pode ser adaptada às necessidades específicas do projeto de *software* e às características da equipe.

Conjuntos de tarefas



Um conjunto de tarefas define o trabalho a ser feito para atingir os objetivos de uma ação de engenharia de software. Por exemplo, levantamento (mais comumente denominado “levantamento de requisitos”) é uma importante ação de engenharia de *software* que ocorre durante a atividade de **comunicação**. A meta do levantamento de requisitos é compreender o que os vários envolvidos esperam do *software* a ser desenvolvido.

Para um projeto pequeno e relativamente simples, o conjunto de tarefas para levantamento dos requisitos seria semelhante a este:

5. Fazer uma lista dos envolvidos no projeto.
8. Fazer uma reunião informal com todos os envolvidos.
7. Solicitar a cada envolvido uma lista com as características e funções necessárias.
0. Discutir sobre os requisitos e elaborar uma lista final.
9. Organizar os requisitos por grau de prioridade.
2. Destacar pontos de incertezas.

Para um projeto de *software* maior e mais complexo, é preciso usar um conjunto diferente de tarefas. Esse conjunto pode incluir as seguintes tarefas de trabalho:

5. Fazer uma lista dos envolvidos no projeto.
8. Entrevistar separadamente cada um dos envolvidos para levantamento geral de suas expectativas e necessidades.
7. Fazer uma lista preliminar das funções e características, com base nas informações fornecidas pelos envolvidos.
0. Agendar uma série de reuniões facilitadoras para especificação de aplicações.
9. Realizar reuniões.
2. Incluir cenários informais de usuários como parte de cada reunião.
1. Refinar os cenários de usuários, com base no *feedback* dos envolvidos.
4. Fazer uma lista revisada dos requisitos dos envolvidos.
3. Empregar técnicas de implantação de funções de qualidade para estabelecer graus de prioridade dos requisitos.
56. Agrupar os requisitos de modo que possam ser entregues em incrementos.

55. Fazer um levantamento das limitações e restrições que serão aplicadas ao sistema.

58. Discutir sobre os métodos para validação do sistema.

Esses dois conjuntos de tarefas atingem o objetivo do “levantamento de requisitos”; porém, são bem diferentes quanto ao seu grau de profundidade e formalidade. A equipe de *software* deve escolher o conjunto de tarefas que possibilite atingir o objetivo de cada ação, mantendo, inclusive, a qualidade e a agilidade.

2.4 Avaliação e aperfeiçoamento de processos

A existência de um processo de *software* não garante que o *software* será entregue dentro do prazo, que estará de acordo com as necessidades do cliente ou que apresentará características técnicas que resultarão em qualidade em longo prazo (Capítulo 15). Os padrões de processo devem ser combinados com uma prática de engenharia de *software* confiável (Parte II deste livro). Além disso, o próprio processo pode ser avaliado para que esteja de acordo com um conjunto de critérios de processo básicos, comprovados como essenciais para uma engenharia de *software* bem-sucedida.¹

Hoje, o consenso entre a maioria dos engenheiros é que os processos e as atividades de *software* devem ser avaliados utilizando medidas numéricas ou análise de *software* (métricas). O seu progresso na jornada em direção a um processo de *software* eficaz definirá o quanto você consegue mensurar a melhoria de forma significativa. O uso de métricas de processos de *software* para avaliar a qualidade do processo é introduzido no Capítulo 17. Uma discussão mais detalhada sobre a avaliação de processos e métodos de melhoria é apresentada no Capítulo 28.

2.5 Modelos de processo prescritivo

Os modelos de processo prescritivo definem um conjunto prescrito de elementos de processo e um fluxo de trabalho de processo previsível. Um modelo de processo prescritivo² concentra-se em estruturar e ordenar o desenvolvimento de *software*. As atividades e tarefas ocorrem sequencialmente, com diretrizes de progresso definidas. Mas os modelos prescritivos são adequados para um mundo do *software* que se alimenta de mudanças? Se rejeitarmos os modelos de processo tradicionais (e a ordem implícita) e os substituirmos por algo menos estruturado, tornaremos impossível atingir a coordenação e a coerência no trabalho de *software*?

Não há respostas fáceis para essas questões, mas existem alternativas disponíveis para os engenheiros de *software*. Nas próximas seções, apresentamos uma visão geral da abordagem dos processos prescritivos, nos quais a ordem e a consistência do projeto são questões predominantes. Chamamos esses processos de “prescritivos” porque prescrevem um conjunto de elementos de processo – atividades metodológicas, ações de engenharia de *software*, tarefas, artefatos, garantia da qualidade e mecanismos de controle de mudanças para cada projeto. Cada modelo de processo também prescreve um fluxo de processo (também denominado *fluxo de trabalho*) – ou seja, a forma pela qual os elementos do processo estão relacionados.

Todos os modelos de processo de *software* podem acomodar as atividades metodológicas genéricas descritas no Capítulo 1; porém, cada um deles dá uma ênfase diferente a essas atividades e define um fluxo de processo que invoca cada atividade metodológica (bem como tarefas e ações de engenharia de *software*) de forma diversa. Nos Capítulos 3 e 4, analisamos práticas de engenharia de *software* que buscam acomodar as alterações inevitáveis que surgem durante o desenvolvimento de muitos projetos de *software*.

2.5.1 O modelo cascata

Há casos em que os requisitos de um problema são bem compreendidos – quando o trabalho flui da comunicação à entrega de modo relativamente linear. Essa situação ocorre algumas vezes quando adaptações ou aperfeiçoamentos bem-definidos precisam ser feitos em um sistema existente (p. ex., uma adaptação em *software* contábil exigida devido a mudanças nas normas governamentais). Pode ocorrer também em um número limitado de novos esforços de desenvolvimento, mas apenas quando os requisitos estão bem definidos e são razoavelmente estáveis.

O *modelo cascata*, algumas vezes chamado de *modelo sequencial linear*, sugere uma abordagem sequencial e sistemática³ para o desenvolvimento de *software*, começando com a especificação dos requisitos do cliente, avançando pelas fases de planejamento, modelagem, construção e entrega, e culminando no suporte contínuo do *software* concluído (Figura 2.3).

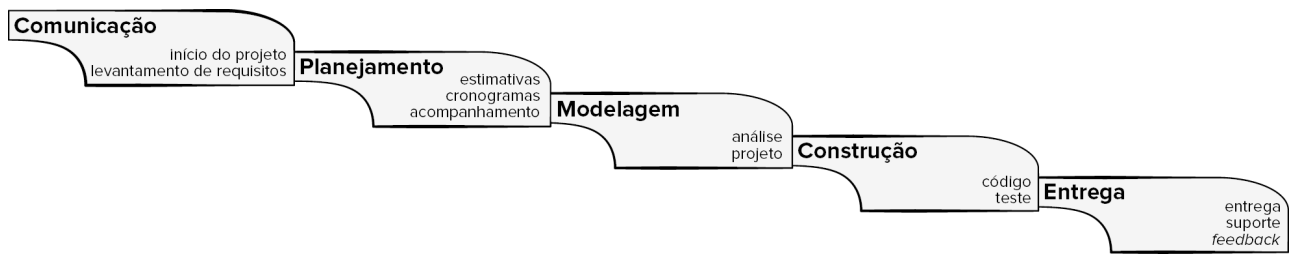


Figura 2.3
O modelo cascata.

O modelo cascata é o paradigma mais antigo da engenharia de *software*. Entretanto, ao longo das últimas cinco décadas, as críticas a esse modelo de processo fizeram até mesmo seus mais áduos defensores questionarem sua eficácia. Entre os problemas às vezes encontrados quando se aplica o modelo cascata, temos:

5. Projetos reais raramente seguem o fluxo sequencial proposto pelo modelo.
8. Com frequência, é difícil para o cliente estabelecer explicitamente todas as necessidades no início da maioria dos projetos.
7. O cliente deve ter paciência. Uma versão operacional do(s) programa(s) não estará disponível antes de estarmos próximos ao final do projeto.
0. Erros graves podem não ser detectados até o programa operacional ser revisto.

Hoje, o trabalho com *software* tem um ritmo acelerado e está sujeito a uma cadeia de mudanças intermináveis (em características, funções e conteúdo de informações). O modelo cascata é frequentemente inadequado para esse trabalho.

2.5.2 Modelo de processo de prototipação

Frequentemente, o cliente define uma série de objetivos gerais para o *software*, mas não identifica, com detalhes, os requisitos para funções e recursos. Em outros casos, o desenvolvedor se encontra inseguro quanto à eficiência de um algoritmo, quanto à adaptabilidade de um sistema operacional ou quanto à forma em que deve ocorrer a interação homem-máquina. Em situações como essas, e em muitas outras, o *paradigma da prototipação* pode ser a melhor abordagem.

Embora a prototipação possa ser utilizada como um modelo de processo isolado (*stand-alone process*), ela é mais comumente utilizada como uma técnica a ser implementada no contexto de qualquer um dos modelos de processo citados neste capítulo. Independentemente da forma como é aplicado, quando os requisitos estão obscuros, o paradigma da prototipação auxilia os envolvidos a compreender melhor o que está para ser construído.

Por exemplo, um aplicativo de *fitness* desenvolvido usando protótipos incrementais poderia oferecer as telas básicas da interface do usuário necessárias para sincronizar um telefone ao dispositivo de *fitness* e mostrar a data atual; a capacidade de definir metas e armazenar os dados do dispositivo na nuvem seriam incluídas no segundo protótipo, que criaria e modificaria as telas da interface do usuário com base no *feedback* dos clientes; e o terceiro protótipo incluiria integração com mídias sociais para permitir que os usuários definissem metas e compartilhassem o seu progresso com os amigos.

O paradigma da prototipação (Figura 2.4) começa com a comunicação. Faz-se uma reunião com os envolvidos para definir os objetivos gerais do *software*, identificar os requisitos já conhecidos e esquematizar quais áreas necessitam, obrigatoriamente, de uma definição mais ampla. Uma iteração de prototipação é planejada rapidamente e ocorre a modelagem (na forma de um “projeto rápido”). Um projeto rápido se concentra em uma representação dos aspectos do *software* que serão visíveis para os usuários (p. ex., o *layout* da interface com o usuário ou os formatos de exibição na tela). O projeto rápido leva à construção de um protótipo. O protótipo é entregue e avaliado pelos envolvidos, os quais fornecem *feedback* que é usado para refinar ainda mais os requisitos. A iteração ocorre conforme se ajusta o protótipo às necessidades de vários envolvidos e, ao mesmo tempo, possibilita a melhor compreensão das necessidades que devem ser atendidas.

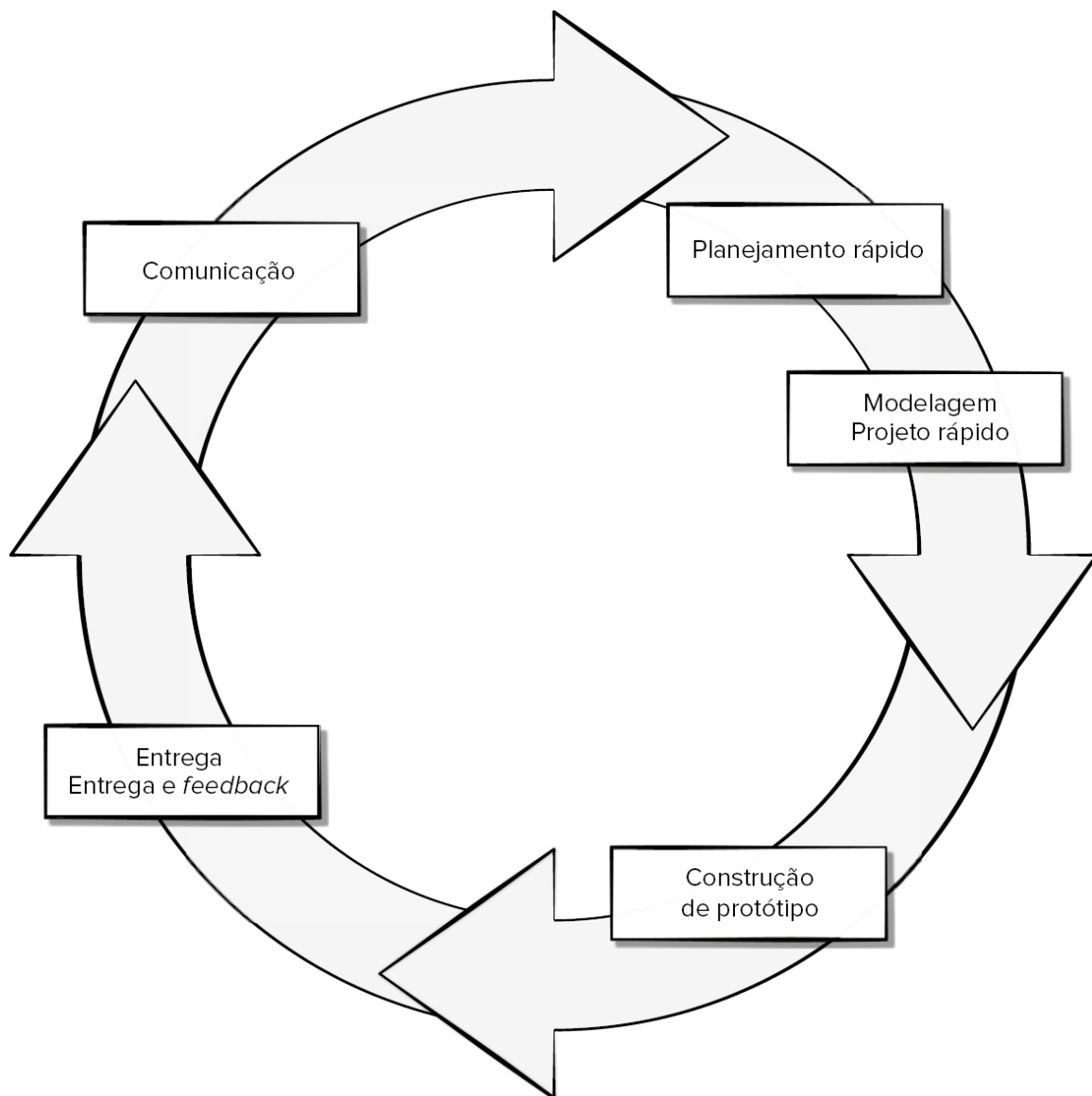


Figura 2.4
O paradigma da prototipação.

Na sua forma ideal, o protótipo atua como um mecanismo para identificar os requisitos do *software*. Caso seja necessário desenvolver um protótipo operacional, pode-se utilizar partes de programas existentes ou aplicar ferramentas que possibilitem gerar rapidamente tais programas operacionais.

Tanto os envolvidos quanto os engenheiros de *software* gostam do paradigma da prototipação. Os usuários podem ter uma ideia prévia do sistema final, ao passo que os desenvolvedores passam a desenvolver algo imediatamente. Entretanto, a prototipação pode ser problemática pelas seguintes razões:

5. Os envolvidos enxergam o que parece ser uma versão operacional do *software*. Eles podem desconhecer que a arquitetura do protótipo (a estrutura do programa) também está evoluindo. Isso significa que os desenvolvedores podem não ter considerado a qualidade global do *software*, nem sua manutenção em longo prazo.
8. O engenheiro de *software* pode ficar tentado a fazer concessões na implementação para conseguir que o protótipo entre em operação rapidamente. Se não tomar cuidado, essas escolhas longe de ideais acabam se tornando uma parte fundamental do sistema.

Casa segura



Seleção de um modelo de processo, Parte 1

Cena: Sala de reuniões da equipe de engenharia de *software* da CPI Corporation, empresa (fictícia) que fabrica produtos de consumo para uso doméstico e comercial.

Atores: Lee Warren, gerente de engenharia; Doug Miller, gerente de engenharia de software; Jamie Lazar, membro da equipe de software; Vinod Raman, membro da equipe de software; e Ed Robbins, membro da equipe de software.

Conversa:

Lee: Recapitulando. Discuti bastante sobre a linha de produtos CasaSegura, da forma como a visualizamos no momento. Sem dúvida, temos muito trabalho a fazer para definir as coisas, mas eu gostaria que vocês começassem a pensar em como vão abordar a parte do *software* desse projeto.

Doug: Acho que fomos bastante desorganizados em nossa abordagem de *software* no passado.

Ed: Eu não sei, Doug, nós sempre conseguimos entregar o produto.

Doug: É, mas não sem grande sofrimento, e esse projeto parece ser maior e mais complexo do que qualquer outro que já fizemos.

Jamie: Não parece assim tão difícil, mas eu concordo... a abordagem improvisada que adotamos em projetos anteriores não dará certo neste caso, principalmente se tivermos um cronograma muito apertado.

Doug (sorrindo): Quero ser um pouco mais profissional em nossa abordagem. Participei de um curso rápido na semana passada e aprendi bastante sobre engenharia de software... bom conteúdo. Precisamos de um processo aqui.

Jamie (franzindo a testa): Minha função é desenvolver programas, não ficar mexendo em papéis.

Doug: Dê uma chance antes de dizer não. Eis o que quero dizer. (Doug prossegue descrevendo a metodologia de processo descrita no Capítulo 1 e os modelos de processo prescritivo apresentados até agora.)

Doug: De qualquer forma, parece-me que um modelo linear não é adequado para nós... ele presume que temos todos os requisitos antecipadamente e, conhecendo este lugar, isso é pouco provável.

Vinod: Isso mesmo, e parece orientado demais à tecnologia da informação... provavelmente bom para construir um sistema de controle de estoque ou algo parecido, mas certamente não é adequado para o CasaSegura.

Doug: Concordo.

Ed: Essa abordagem de prototipação me parece boa. Bastante parecida com o que fazemos aqui.

Vinod: Isso é um problema. Estou preocupado que ela não nos dê estrutura suficiente.

Doug: Não se preocupe. Temos várias opções e quero que vocês escolham o que for melhor para a equipe e para o projeto.

Embora possam ocorrer problemas, a prototipação pode ser um paradigma eficiente para a engenharia de *software*. O segredo é definir as regras do jogo logo no início; ou seja, todos os envolvidos devem concordar que o protótipo é construído para servir como um mecanismo para definição de requisitos. Muitas vezes, é recomendável projetar um protótipo de modo que este possa evoluir e se transformar no produto final. A realidade é que os desenvolvedores podem precisar descartar (pelo menos em parte) um protótipo para melhor atender às novas necessidades do cliente.

2.5.3 Modelo de processo evolucionário

Como todos os sistemas complexos, o *software* evolui ao longo do tempo. Conforme o desenvolvimento do projeto avança, os requisitos do negócio e do produto frequentemente mudam, tornando inadequado seguir um planejamento em linha reta de um produto final. Prazos apertados tornam impossível concluir um produto de *software* abrangente. Pode ser possível criar uma versão limitada do produto para atender às pressões comerciais ou da concorrência e lançar uma versão refinada após um melhor entendimento de todas as características do sistema. Em situações como essa, faz-se necessário um modelo de processo que tenha sido projetado especificamente para desenvolver um produto que cresce e muda.

Originalmente proposto por Barry Boehm [Boe88], o *modelo espiral* é um modelo de processo de *software* evolucionário que une a natureza iterativa da prototipação aos aspectos sistemáticos e controlados do modelo cascata. Tem potencial para o rápido desenvolvimento de versões cada vez mais completas do *software*.

Com o modelo espiral, o *software* será desenvolvido em uma série de versões evolucionárias. Nas primeiras iterações, a versão pode consistir em um modelo ou em um protótipo. Já nas iterações posteriores, são produzidas versões cada vez mais completas do sistema que passa pelo processo de engenharia.

O modelo espiral é dividido em um conjunto de atividades metodológicas definidas pela equipe de engenharia de *software*. A título de ilustração, utilizam-se as atividades metodológicas genéricas discutidas anteriormente.⁴ Cada uma dessas atividades representa um segmento do caminho espiral

ilustrado na Figura 2.5. Assim que esse processo evolucionário começa, a equipe de *software* realiza atividades indicadas por um circuito em torno da espiral, no sentido horário, começando pelo seu centro. Os riscos (Capítulo 26) são levados em conta à medida que cada revolução é realizada. *Marcos de pontos-âncora* – uma combinação de artefatos e condições satisfeitas ao longo do trajeto da espiral – são indicados para cada passagem evolucionária.

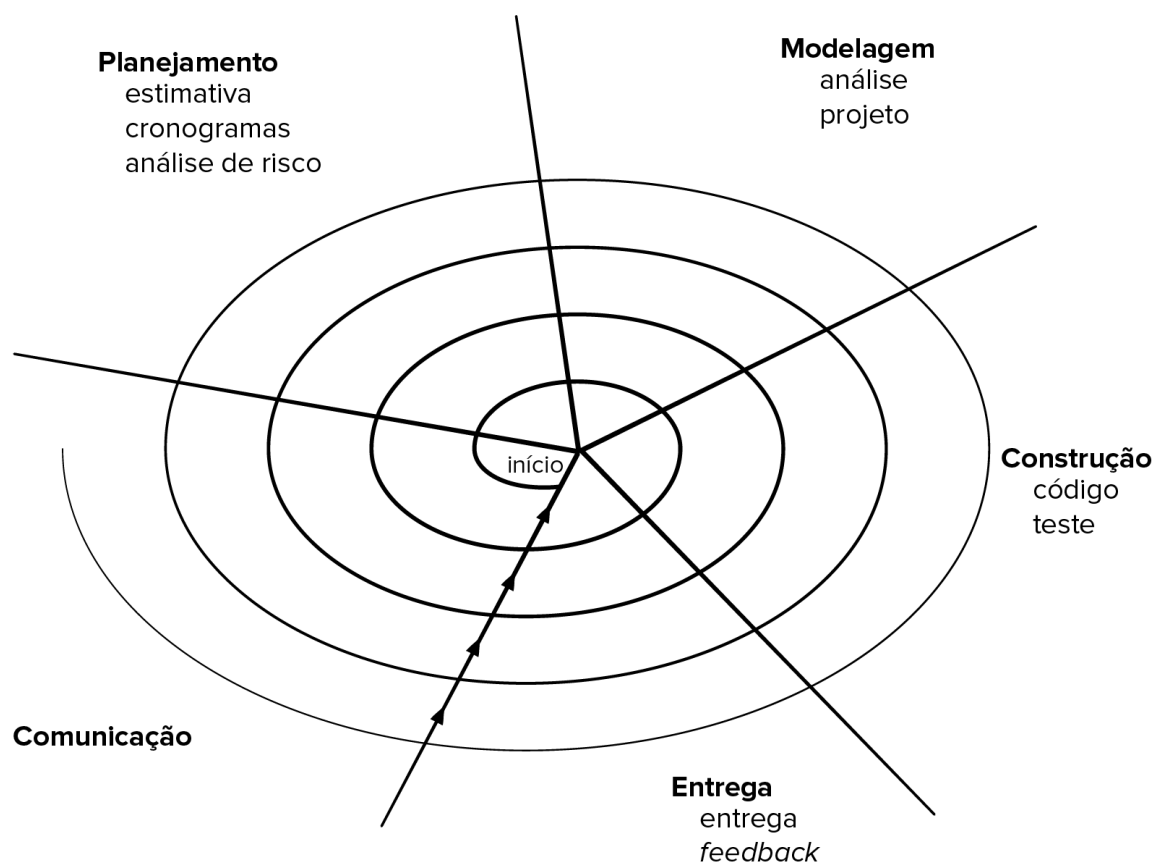


Figura 2.5
Modelo espiral típico.

O primeiro circuito em volta da espiral (iniciando na linha de fluxo interna mais próxima ao centro, como mostrado na Figura 2.5) pode resultar no desenvolvimento de uma especificação de produto; passagens subsequentes em torno da espiral podem ser usadas para desenvolver um protótipo e, então, progressivamente, versões cada vez mais sofisticadas do *software*. Cada passagem pela região de planejamento resulta em ajustes no planejamento do projeto. Custo e cronograma são ajustados de acordo com o *feedback* (a realimentação) obtido do cliente após a entrega. Além disso, o gerente de projeto faz um ajuste no número de iterações planejadas para concluir o *software*.

Diferentemente de outros modelos de processo, que terminam quando o *software* é entregue, o modelo espiral pode ser adaptado para ser aplicado ao longo da vida do *software*. O modelo espiral é uma abordagem realista para o desenvolvimento de sistemas e de *software* em larga escala. Ele usa a prototipação como mecanismo de redução de riscos. O modelo espiral exige consideração direta dos riscos técnicos em todos os estágios do projeto e, se aplicado apropriadamente, reduz os riscos antes destes se tornarem problemáticos.

Como outros paradigmas, esse modelo não é uma panaceia. Pode ser difícil convencer os clientes (particularmente em situações contratuais) de que a abordagem evolucionária é controlável. Ela exige considerável especialização na avaliação de riscos e depende dessa especialização para seu sucesso. Se um risco muito importante não for descoberto e administrado, sem dúvida ocorrerão problemas.

Conforme já mencionado, *software* moderno é caracterizado por contínuas modificações, prazos muito apertados e por uma ênfase na satisfação do cliente-usuário. Em muitos casos, o tempo de colocação de um produto no mercado é o requisito mais importante a ser gerenciado. Se o momento oportuno de entrada no mercado for perdido, o projeto de *software* pode perder o sentido.⁵



Seleção de um modelo de processo, Parte 2

Cena: Sala de reuniões do grupo de engenharia de *software* da CPI Corporation, empresa (fictícia) que fabrica produtos de consumo de uso doméstico e comercial.

Atores: Lee Warren, gerente de engenharia; Doug Miller, gerente de engenharia de *software*; Vinod e Jamie, membros da equipe de engenharia de *software*.

Conversa: (Doug descreve as opções do processo evolucionário.)

Jamie: Agora estou vendo algo de que gosto. Faz sentido uma abordagem incremental, e eu realmente gosto do fluxo dessa coisa de modelo espiral. Isso tem a ver com a realidade.

Vinod: Concordo. Entregamos um incremento, aprendemos com o *feedback* do cliente, reformulamos e, então, entregamos outro incremento. Também se encaixa na natureza do produto. Podemos colocar alguma coisa no mercado rapidamente e, depois, acrescentar funcionalidade a cada versão, digo, incremento.

Lee: Espere um pouco. Você disse que reformulamos o plano a cada volta na espiral, Doug? Isso não é tão legal; precisamos de um plano, um cronograma, e temos de nos ater a ele.

Doug: Essa linha de pensamento é antiga, Lee. Como o pessoal disse, temos de manter os pés no chão. Acho que é melhor ir ajustando o planejamento à medida que formos aprendendo mais e as mudanças forem sendo solicitadas. É muito mais realista. Para que serve um plano se não para refletir a realidade?

Lee (franzindo a testa): Suponho que esteja certo, porém... a alta direção não vai gostar disso... eles querem um plano fixo.

Doug (sorrindo): Então, você terá que reeducá-los, meu amigo.

O objetivo dos modelos evolucionários é desenvolver *software* de alta qualidade⁶ de modo iterativo ou incremental. Entretanto, é possível usar um processo evolucionário para enfatizar a flexibilidade, a extensibilidade e a velocidade de desenvolvimento. O desafio para as equipes de *software* e seus gerentes será estabelecer um equilíbrio apropriado entre esses parâmetros críticos de projeto e produto e a satisfação dos clientes (o árbitro final da qualidade de um *software*).

2.5.4 Modelo de Processo Unificado

Sob certos aspectos, o Processo Unificado (PU) [Jac99] é uma tentativa de aproveitar os melhores recursos e características dos modelos tradicionais de processo de *software*, mas caracterizando-os de modo a implementar muitos dos melhores princípios do desenvolvimento ágil de *software* (Capítulo 3). O Processo Unificado reconhece a importância da comunicação com o cliente e de métodos racionalizados para descrever a visão do cliente sobre um sistema (os casos de uso).⁷ Ele enfatiza o importante papel da arquitetura de *software* e “ajuda o arquiteto a manter o foco nas metas corretas, como compreensibilidade, confiança em mudanças futuras e reutilização” [Jac99]. Ele sugere um fluxo de processo iterativo e incremental, proporcionando a sensação evolucionária que é essencial no desenvolvimento de *software* moderno.

A UML (*unified modeling language*), a *linguagem de modelagem unificada*, foi desenvolvida para apoiar o seu trabalho, contém uma notação robusta para a modelagem e o desenvolvimento de sistemas orientados a objetos e tornou-se um padrão da indústria para o desenvolvimento de *software* de todos os tipos. A UML é usada ao longo da Parte II deste livro para representar tanto modelos de projeto quanto de requisitos. O Apêndice 1 apresenta um tutorial introdutório e uma bibliografia recomendada para aqueles que não conhecem as regras básicas de notações e de modelagem UML.

A Figura 2.6 representa as “fases” do Processo Unificado e relaciona-as às atividades genéricas discutidas na Seção 2.1.

Na *fase de concepção* do PU ocorre a comunicação com o cliente e o planejamento. Requisitos de negócio fundamentais são descritos em um conjunto de casos de uso preliminares (Capítulo 7), descrevendo quais recursos e funções são esperados na arquitetura de *software* por cada categoria principal de usuário. O planejamento identifica recursos, avalia os principais riscos e define um cronograma preliminar para os incrementos de *software*.

A *fase de elaboração* inclui as atividades de planejamento e modelagem do modelo de processo genérico (Figura 2.6). A elaboração refina e expande os casos de uso preliminares e cria uma base de arquitetura, incluindo cinco diferentes visões do *software*: modelo de caso de uso, modelo de análise, modelo de projeto, modelo de implementação e modelo de entrega.⁸ Normalmente, as modificações no planejamento são feitas nesta fase.

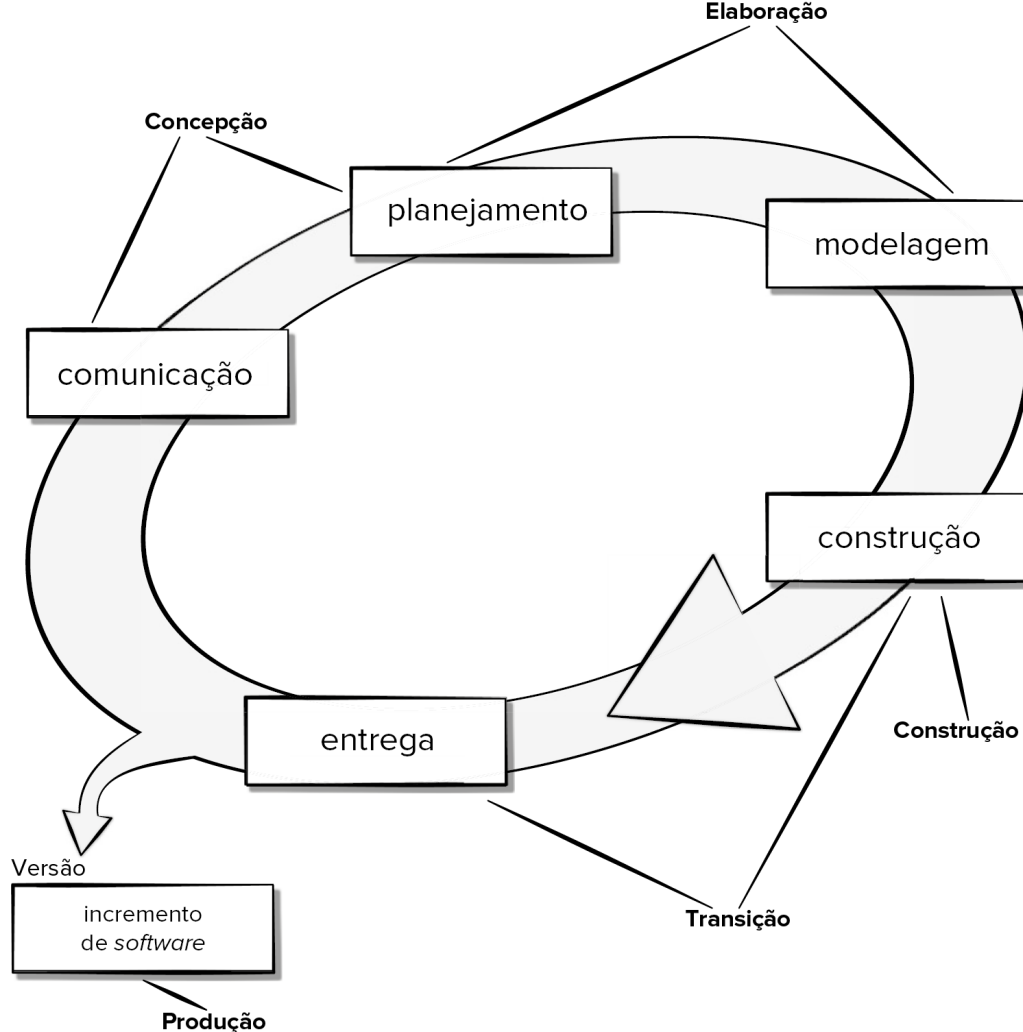


Figura 2.6
O Processo Unificado.

A *fase de construção* do PU é idêntica à atividade de construção definida para o processo de *software* genérico. Implementam-se, então, no código-fonte, todos os recursos e funções necessários e exigidos para o incremento de *software* (i.e., para a versão). À medida que os componentes são implementados, desenvolvem-se e executam-se testes de unidade⁹ para cada um deles. Além disso, realizam-se atividades de integração (montagem de componentes e testes de integração). Os casos de uso são utilizados para se obter um pacote de testes de aceitação, executados antes do início da fase seguinte do PU.

A *fase de transição* do PU abrange os últimos estágios da atividade de construção genérica e a primeira parte da atividade de emprego genérico: entrega e *feedback*. Entrega-se o *software* aos usuários para testes beta, e o *feedback* dos usuários relata defeitos e mudanças necessárias. Na conclusão da fase de transição, o incremento torna-se uma versão utilizável do *software*.

A *fase de produção* do PU coincide com a atividade de entrega do processo genérico. Durante esta fase, monitora-se o uso contínuo do *software*, disponibiliza-se suporte para o ambiente (infraestrutura) operacional, realizam-se e avaliam-se relatórios de defeitos e solicitações de mudanças.

É provável que, ao mesmo tempo em que as fases de construção, transição e produção estejam sendo conduzidas, já se tenha iniciado o incremento de *software* seguinte. Isso significa que as cinco fases do PU não ocorrem em sequência, mas de forma concomitante e escalonada.

Deve-se observar que nem toda tarefa identificada para um fluxo de trabalho do PU é conduzida em todos os projetos de *software*. A equipe adapta o processo (ações, tarefas, subtarefas e artefatos) para ficar de acordo com suas necessidades.

2.6 Produto e processo

A Tabela 2.1 resume alguns dos pontos fortes e fracos dos modelos de processo analisados. Nas edições anteriores deste livro, examinamos vários outros. A realidade é que nenhum processo é perfeito para todos os projetos. Em geral, a equipe de *software* adapta um ou mais dos modelos de processo analisados na Seção 2.5 ou os modelos de processo ágil discutidos no Capítulo 3 para atender às necessidades do projeto do momento.

Tabela 2.1 Comparação entre modelos de processo

Prós do modelo cascata	É fácil de entender e planejar. Funciona para projetos pequenos e bem compreendidos. A análise e o teste são simples e diretos.
Contras do modelo cascata	Não se adapta bem a mudanças. O teste ocorre nas fases finais do processo. A aprovação do cliente vem no final.
Prós da prototipação	O impacto das alterações aos requisitos é reduzido. O cliente se envolve bastante e desde o início. Funciona bem para projetos pequenos. A probabilidade de rejeição do produto é reduzida.
Contras da prototipação	O envolvimento do cliente pode causar atrasos. Pode haver a tentação de “embalar” o protótipo. Desperdiça-se trabalho em um protótipo descartável. É difícil de planejar e gerenciar.
Prós do modelo espiral	Há envolvimento contínuo dos clientes. Os riscos de desenvolvimento são gerenciados. É apropriado para modelos grandes e complexos. Funciona bem para artefatos extensíveis.
Contras do modelo espiral	Falhas de análise de risco podem fadar o projeto ao fracasso. O projeto pode ser difícil de gerenciar. Exige uma equipe de desenvolvimento especializada.
Prós do Processo Unificado	A documentação de alta qualidade é enfatizada. Há envolvimento contínuo dos clientes. Adapta-se a alterações aos requisitos. Funciona bem para projetos de manutenção.
Contras do Processo Unificado	Os casos de uso nem sempre são precisos. A integração de incrementos de <i>software</i> é complicada. A sobreposição das fases pode causar problemas. Exige uma equipe de desenvolvimento especializada.

Se o processo for fraco, certamente o produto final sofrerá consequências. Porém, uma confiança excessiva e obsessiva no processo é igualmente perigosa. Em um breve artigo, escrito muitos anos atrás, Margaret Davis [Dav95a] tece comentários atemporais sobre a dualidade produto e processo:

A cada 10 anos (com uma margem de erro de cinco anos), aproximadamente, a comunidade de *software* redefine “o problema”, mudando seu foco de itens do produto para itens do processo. (...)

Embora a tendência natural de um pêndulo seja a de repousar em um ponto intermediário entre dois extremos, o foco da comunidade de *software* muda constantemente, pois uma nova força é aplicada quando a última oscilação falha. Essas oscilações causam danos para si mesmos e para o ambiente externo, confundindo o profissional típico de *software*, mudando radicalmente o que significava desempenhar bem seu trabalho. Essas oscilações também não resolvem “o problema”, pois estão fadadas ao insucesso enquanto produto e processo forem tratados como formadores de uma dicotomia (divisão de um conceito em dois elementos, em geral, contrários) em vez de uma dualidade (coexistência de dois princípios).

(...) Jamais poderemos destrinchar ou compreender o artefato completo, seu contexto, uso, significado e valor se o enxergarmos apenas como um processo ou como um produto.

Todas as atividades humanas podem ser um processo, mas todos nós nos sentimos valorizados quando tais atividades se tornam uma representação ou um exemplo, sendo utilizadas ou apreciadas por mais de uma pessoa, repetidamente, ou então utilizadas num contexto não imaginado. Ou seja, extraímos sentimentos de satisfação na reutilização de nossos produtos, seja por nós mesmos, seja por outros.

Assim, enquanto a assimilação rápida das metas de reutilização no desenvolvimento de *software* aumenta potencialmente a satisfação dos profissionais de *software*, também aumenta a urgência da aceitação da dualidade produto e processo. (...)

As pessoas obtêm satisfação tanto do processo criativo quanto do produto final. Um artista sente prazer tanto com suas pinceladas quanto com o resultado final de seu quadro. Um escritor sente prazer tanto com a procura da metáfora apropriada quanto com o livro finalizado. Como profissional de *software* criativo, você também deve extrair satisfação do processo e do produto final. A dualidade produto e processo é um elemento importante para manter pessoas criativas engajadas, à medida que a engenharia de *software* continua a evoluir.

2.7 Resumo

Um modelo de processo genérico para engenharia de *software* consiste em um conjunto de atividades metodológicas e de apoio, ações e tarefas a realizar. Cada modelo de processo, entre os vários existentes, pode ser descrito por um fluxo de processo diferente – uma descrição de como as atividades metodológicas, ações e tarefas são organizadas, sequencial e cronologicamente. Padrões de processo são utilizados para resolver problemas comuns encontrados no processo de *software*.

Os modelos de processo prescritivos são aplicados há anos, na tentativa de organizar e estruturar o desenvolvimento de *software*. Cada um desses modelos sugere um fluxo de processo um pouco diferente, mas todos realizam o mesmo conjunto de atividades metodológicas genéricas: comunicação, planejamento, modelagem, construção e entrega.

Os modelos de processo sequenciais, como o modelo cascata, são os mais antigos paradigmas da engenharia de *software*. Eles sugerem um fluxo de processo linear que, muitas vezes, é inadequado para os sistemas modernos (p. ex., alterações contínuas, sistemas em evolução, prazos apertados). Entretanto, eles têm aplicabilidade em situações em que os requisitos são bem definidos e estáveis.

Modelos de processo incremental são iterativos por natureza e produzem rapidamente versões operacionais do *software*. Modelos de processo evolucionário reconhecem a natureza iterativa e incremental da maioria dos projetos de engenharia de *software* e são projetados para se adequar às mudanças. Esses modelos, como prototipação e o modelo espiral, produzem rapidamente artefatos de *software* incrementais (ou versões operacionais do *software*). Eles podem ser aplicados em todas as atividades de engenharia de *software* – desde o desenvolvimento de conceitos até a manutenção do sistema em longo prazo.

O Processo Unificado é um processo de *software* “dirigido a casos de uso, centrado na arquitetura, iterativo e incremental” desenvolvido como uma metodologia para os métodos e ferramentas da UML.

Problemas e pontos a ponderar

- 2.1 Baetjer [Bae98] observa: “O processo oferece interação entre usuários e projetistas, entre usuários e ferramentas em evolução e entre projetistas e ferramentas [de tecnologia] em evolução”. Liste cinco perguntas que (1) os projetistas deveriam fazer aos usuários, (2) os usuários deveriam fazer aos projetistas, (3) os usuários deveriam fazer a si mesmos sobre o produto de *software* a ser desenvolvido, (4) os projetistas deveriam fazer a si mesmos sobre o produto de *software* a ser construído e sobre o processo que será usado para construí-lo.
- 2.2 Discuta as diferenças entre os vários fluxos de processo descritos na Seção 2.1. Identifique os tipos de problemas que poderiam ser aplicáveis a cada um dos fluxos genéricos descritos.
- 2.3 Tente desenvolver um conjunto de ações para a atividade de comunicação. Selecione uma ação e defina um conjunto de tarefas para ela.
- 2.4 Durante a comunicação, um problema comum ocorre ao encontrarmos dois envolvidos com ideias conflitantes sobre como o *software* deve ser. Isto é, há requisitos mutuamente conflitantes. Desenvolva um padrão de processo que trata desse problema e sugira uma abordagem eficaz para ele.
- 2.5 Dê três exemplos de projetos de *software* que seriam suscetíveis ao modelo cascata. Seja específico.
- 2.6 Dê três exemplos de projetos de *software* que seriam suscetíveis ao modelo de prototipação. Seja específico.
- 2.7 À medida que se desloca para fora ao longo do fluxo de processo em espiral, o que pode ser dito em relação ao *software* que está sendo desenvolvido ou sofrendo manutenção?
- 2.8 É possível combinar modelos de processo? Em caso positivo, dê um exemplo.
- 2.9 Quais são as vantagens e desvantagens de desenvolver *software* cuja qualidade é “boa o suficiente”? Ou seja, o que acontece quando enfatizamos a velocidade de desenvolvimento em detrimento da qualidade do produto?
- 2.10 É possível provar que um componente de *software* e até mesmo um programa inteiro está correto? Então, por que todo mundo não faz isso?
- 2.11 Processo Unificado e UML são a mesma coisa? Justifique sua resposta.

1 A CMMI-DEV [CMM07] da SEI descreve, de forma extremamente detalhada, as características de um processo de *software* e os critérios para o êxito de um processo.

2 Os modelos de processo prescritivo são, algumas vezes, conhecidos como modelos de processo “tradicional”.

3 Embora o modelo cascata proposto por Winston Royce [Roy70] previsse os “*feedback loops*”, a vasta maioria das organizações que aplica esse modelo de processo o trata como se fosse estritamente linear.

- 4** O modelo espiral discutido nesta seção é uma variação do modelo proposto por Boehm. Para mais informações sobre o modelo espiral original, consulte [Boe88]. Um material mais recente sobre o modelo espiral de Boehm pode ser encontrado em [Boe98] e [Boe01a].
- 5** É importante notar, entretanto, que ser o primeiro a chegar ao mercado não é garantia de sucesso. Na verdade, muitos produtos de *software* bem-sucedidos foram o segundo ou até mesmo o terceiro a chegar ao mercado (aprendendo com os erros dos outros que o antecederam).
- 6** Neste contexto, a qualidade de *software* é definida de forma bastante abrangente para englobar não apenas a satisfação dos clientes, mas também uma série de critérios técnicos, discutidos na Parte II deste livro.
- 7** Um *caso de uso* (Capítulo 7) é uma narrativa textual ou um modelo que descreve uma função ou recurso de um sistema do ponto de vista do usuário. Ele é escrito pelo usuário e serve como base para a criação de um modelo de análise mais amplo.
- 8** É importante observar que a base da arquitetura não é um protótipo, já que não é descartada. Ao contrário, a base ganha corpo durante a fase seguinte do PU.
- 9** Uma discussão abrangente sobre testes de *software* (inclusive *testes de unidades*) é apresentada nos Capítulos 19 a 21.

Esta página foi obtida através do recurso de impressão, disponível na Biblioteca

A. Uso restrito a usuários com acesso a plataforma. Todos direitos reservados ao Grupo A (<https://grupoa.com.br/>)

biblioteca a⁺